

1. Introduction

The Internet of Things (IoT) refers to a rapidly expanding network of interconnected physical devices embedded with sensors, software, and communication capabilities that enable them to collect and exchange data over the internet. These devices range from simple environmental sensors to complex industrial systems and play a critical role in domains such as smart homes, healthcare, agriculture, and industrial automation.

However, the widespread adoption of IoT has introduced significant cybersecurity challenges. Unlike traditional computing systems, IoT devices are often resource-constrained, with limited processing power, memory, and energy capacity. As a result, many devices lack robust security mechanisms, making them vulnerable to a wide range of cyber threats.

One of the primary concerns in IoT cybersecurity is the confidentiality and integrity of transmitted data. Since IoT devices frequently communicate over wireless networks, they are susceptible to attacks such as eavesdropping, data interception, and man-in-the-middle attacks. Unauthorized access to IoT systems can lead to data breaches, system manipulation, and even physical damage in critical applications like healthcare or industrial control systems.

Traditional cryptographic techniques such as symmetric and asymmetric encryption are commonly used to secure communication. However, these methods rely on computational hardness assumptions, which may become vulnerable with the advancement of quantum computing. This has led to increasing interest in quantum-resistant and quantum-based security approaches, such as Quantum Key Distribution (QKD), which leverage the principles of quantum mechanics to ensure secure key exchange.

Another major challenge is device authentication and access control. Many IoT devices are deployed with weak or default credentials, making them easy targets for attackers. Additionally, the large-scale and distributed nature of IoT networks makes centralized security management difficult.

To address these challenges, IoT cybersecurity focuses on multiple layers of protection, including:

- Secure communication protocols
- Strong encryption techniques
- Device authentication and identity management
- Intrusion detection and anomaly monitoring
- Regular firmware updates and patch management

In recent years, there has been a growing emphasis on integrating advanced security mechanisms, including machine learning-based threat detection and quantum-inspired cryptographic systems, to enhance the resilience of IoT networks.

1.1 Introduction to Cryptography

Cryptography is the science of securing information by transforming it into a form that is unintelligible to unauthorized users while remaining accessible to intended recipients. It plays a fundamental role in modern digital communication by ensuring **confidentiality, integrity, authentication, and non-repudiation** of data.

The need for cryptography arises from the increasing reliance on digital systems for communication, storage, and processing of sensitive information. As data travels across networks—especially public or wireless channels—it becomes vulnerable to interception, modification, and unauthorized access. Cryptographic techniques are employed to protect this data from such threats.

At its core, cryptography involves two main processes: **encryption** and **decryption**. Encryption converts plaintext (readable data) into ciphertext (encoded data) using a mathematical algorithm and a key. Decryption is the reverse process, where the ciphertext is converted back into plaintext using the appropriate key. The security of a cryptographic system largely depends on the secrecy and strength of the key rather than the secrecy of the algorithm.

Cryptography can be broadly classified into three categories:

1. Symmetric Key Cryptography

In this method, the same key is used for both encryption and decryption. It is computationally efficient and suitable for resource-constrained systems such as IoT devices. However, securely distributing and managing the shared key remains a major challenge.

2. Asymmetric Key Cryptography

Also known as public-key cryptography, this method uses a pair of keys: a public key for encryption and a private key for decryption. It simplifies key distribution but is computationally more intensive compared to symmetric methods.

3. Hash Functions

These are one-way functions that convert data into a fixed-length hash value. They are primarily used for ensuring data integrity and authentication, as even a small change in input produces a significantly different output.

Despite the effectiveness of traditional cryptographic systems, they rely on mathematical problems that are difficult to solve using classical computers. However, the emergence of quantum computing poses a potential threat to these systems, as quantum algorithms may be capable of breaking widely used encryption techniques.

1.2 Introduction to IoT Data Encryption

The rapid expansion of the Internet of Things (IoT) has led to an unprecedented increase in the volume of data generated, transmitted, and processed by interconnected devices. These devices, ranging from simple sensors to complex industrial systems, continuously exchange information over wired and wireless networks. As a result, ensuring the security of this data has become a critical

concern. Among the various security mechanisms, **data encryption** plays a central role in protecting IoT communications.

IoT data encryption refers to the process of converting readable data (plaintext) into an encoded format (ciphertext) to prevent unauthorized access during storage and transmission. This transformation ensures that even if data is intercepted, it remains unintelligible without the appropriate decryption key. Encryption is essential for maintaining **data confidentiality**, safeguarding sensitive information from malicious actors.

One of the primary challenges in IoT environments is that devices often operate under **resource constraints**, including limited processing power, memory, and energy availability. These limitations make it difficult to implement traditional, computation-heavy encryption algorithms. Therefore, IoT systems require **lightweight encryption techniques** that balance security with efficiency.

Another important aspect of IoT data encryption is the **security of data in transit**. Since IoT devices frequently communicate over wireless channels, they are vulnerable to attacks such as eavesdropping, replay attacks, and man-in-the-middle attacks. Encryption ensures that intercepted data cannot be easily interpreted or modified, thereby preserving both confidentiality and integrity.

IoT encryption can be broadly categorized into two main types:

- **Symmetric Encryption:** Uses a single shared key for both encryption and decryption. It is faster and more suitable for constrained devices but requires secure key distribution.
- **Asymmetric Encryption:** Uses a pair of keys (public and private), providing better key management but at a higher computational cost.

In addition to these, **hybrid encryption approaches** are often employed, combining the efficiency of symmetric encryption with the secure key exchange capabilities of asymmetric methods.

A major challenge in IoT data encryption lies in **key management**. Secure generation, distribution, storage, and updating of encryption keys are critical to maintaining system security. Weak key management can compromise even the strongest encryption algorithms.

With the advancement of computing technologies, particularly **quantum computing**, traditional encryption methods face potential risks due to their reliance on mathematical complexity. This has led to increasing interest in **quantum-resistant and quantum-based encryption techniques**, which aim to provide long-term security against emerging computational threats.

Furthermore, IoT encryption must also consider **scalability and interoperability**, as IoT ecosystems often consist of heterogeneous devices and platforms. Ensuring consistent security across such diverse systems remains an ongoing research challenge.

1.3 Challenges in IoT Security

Despite the rapid growth and widespread adoption of the Internet of Things (IoT), ensuring robust security remains a significant challenge. The unique characteristics of IoT systems—such as large-scale deployment, device heterogeneity, and resource constraints—introduce vulnerabilities that are

not typically present in traditional computing environments. These challenges must be addressed to ensure the safe and reliable operation of IoT networks.

One of the primary challenges is the **resource-constrained nature of IoT devices**. Many IoT nodes are designed to be low-cost and energy-efficient, with limited processing power, memory, and battery life. As a result, implementing strong encryption algorithms and advanced security protocols becomes difficult, often leading to the adoption of weaker or minimal security measures.

Another critical issue is the **lack of standardized security frameworks**. IoT ecosystems consist of devices from different manufacturers, each using varying communication protocols and security mechanisms. This lack of uniform standards leads to inconsistencies in security implementation, making it difficult to ensure end-to-end protection across the network.

Scalability is also a major concern. IoT networks can consist of thousands or even millions of devices, making centralized security management complex. As the number of connected devices increases, so does the attack surface, providing more entry points for potential attackers.

The problem of **weak authentication and access control** further exacerbates security risks. Many IoT devices are deployed with default credentials or lack robust authentication mechanisms, making them vulnerable to unauthorized access. Once compromised, these devices can be used to launch larger attacks, such as distributed denial-of-service (DDoS) attacks.

Another significant challenge is **data privacy and confidentiality**. IoT devices often handle sensitive information, including personal, environmental, and industrial data. Without proper encryption and secure communication protocols, this data can be intercepted, leading to privacy breaches and potential misuse.

Insecure communication channels also pose a major threat. IoT devices frequently rely on wireless communication technologies, which are inherently more vulnerable to attacks such as eavesdropping, replay attacks, and man-in-the-middle attacks. Ensuring secure data transmission over such channels is a critical requirement.

Additionally, **software and firmware vulnerabilities** are common in IoT systems. Many devices do not support regular updates or patches, leaving them exposed to known security flaws. Even when updates are available, they may not be applied consistently due to device limitations or user negligence.

The **physical accessibility of devices** introduces another layer of risk. Unlike traditional IT systems, IoT devices are often deployed in open or remote environments, making them susceptible to physical tampering, hardware manipulation, or unauthorized data extraction.

Emerging technologies, particularly **quantum computing**, present a future challenge to IoT security. Many existing cryptographic algorithms rely on mathematical problems that could potentially be solved efficiently by quantum computers, thereby compromising current encryption methods.

Finally, **energy constraints and real-time requirements** further complicate security implementation. IoT devices often need to operate continuously with minimal power consumption while maintaining low latency, limiting the feasibility of complex security operations.

1.4 Introduction to Quantum Computing

Quantum Computing is an emerging paradigm of computation that leverages the principles of quantum mechanics to process information in fundamentally different ways compared to classical computing. While classical computers use bits as the basic unit of information—represented as either 0 or 1—quantum computers use **quantum bits (qubits)**, which can exist in multiple states simultaneously due to the principle of superposition.

One of the key features of quantum computing is **superposition**, which allows a qubit to represent both 0 and 1 at the same time. This property enables quantum computers to perform many calculations in parallel, significantly increasing computational efficiency for certain types of problems. Another important concept is **entanglement**, a phenomenon where two or more qubits become correlated in such a way that the state of one qubit is dependent on the state of another, regardless of the distance between them. This enables highly coordinated computations that are not possible in classical systems.

Quantum computing also utilizes **quantum interference**, which allows certain computational paths to be amplified while others are canceled out, improving the probability of obtaining correct results. These principles together enable quantum computers to solve specific complex problems—such as factorization, optimization, and simulation of quantum systems—more efficiently than classical computers.

Despite its potential advantages, quantum computing introduces significant challenges to existing security systems. Many widely used cryptographic algorithms rely on the computational difficulty of problems such as integer factorization and discrete logarithms. Quantum algorithms, such as Shor's algorithm, have the theoretical capability to solve these problems efficiently, posing a threat to current encryption standards.

At the same time, quantum computing has also given rise to new approaches in secure communication, particularly in the field of **quantum cryptography**. Techniques such as Quantum Key Distribution (QKD) use the fundamental laws of quantum mechanics to enable secure key exchange, where any attempt at eavesdropping can be detected.

Although large-scale, practical quantum computers are still under development, ongoing research and advancements continue to bring this technology closer to reality. As a result, understanding quantum computing is essential for developing future-proof security mechanisms, especially in domains like IoT where secure data transmission is critical.

1.5 Introduction to Quantum Cryptography

Quantum Cryptography is an advanced field of secure communication that applies the principles of quantum mechanics to protect information. Unlike classical cryptographic systems, which rely on mathematical complexity for security, quantum cryptography derives its strength from the fundamental laws of physics. This makes it inherently resistant to many forms of computational attacks, including those posed by future quantum computers.

The most prominent application of quantum cryptography is **Quantum Key Distribution (QKD)**, a technique used to securely generate and share cryptographic keys between two parties. QKD ensures that any attempt to intercept or measure the transmitted quantum information will inevitably disturb the system, thereby revealing the presence of an eavesdropper. This property is based on key quantum principles such as **superposition** and the **observer effect**, where measurement alters the state of a quantum system.

In traditional encryption systems, the security of communication depends on the secrecy of the key and the computational difficulty of breaking the encryption algorithm. However, these systems are vulnerable to advances in computational power, particularly with the development of quantum computing. In contrast, quantum cryptography provides **information-theoretic security**, meaning that its security does not depend on computational assumptions but on the physical behavior of quantum particles.

A widely studied QKD protocol is the **BB84 protocol**, which uses quantum states of photons to encode binary information. In this protocol, the sender transmits qubits encoded in different bases, and the receiver measures them using randomly chosen bases. After transmission, both parties compare their measurement bases over a classical channel and retain only the matching results to form a shared secret key. Any discrepancies in the key can indicate the presence of an eavesdropper.

Quantum cryptography also addresses one of the fundamental challenges in secure communication: **key distribution**. While classical methods require secure channels or complex key exchange mechanisms, QKD enables secure key sharing even over potentially insecure communication channels.

Despite its strong security guarantees, quantum cryptography faces practical challenges, including the need for specialized hardware, limited transmission distance, and sensitivity to noise and signal loss. Additionally, large-scale deployment remains an area of ongoing research and development.

1.6 Comparison Between Classical Cryptography and Quantum Cryptography

Cryptography is essential for securing communication systems, but the approaches used in classical and quantum cryptography differ fundamentally in their principles, implementation, and security guarantees. While classical cryptography relies on mathematical algorithms and computational difficulty, quantum cryptography is based on the laws of quantum mechanics, offering a new paradigm for secure communication.

1. Fundamental Principle

Classical cryptography is based on mathematical problems that are computationally hard to solve, such as integer factorization and discrete logarithms. Its security depends on the assumption that these problems cannot be solved efficiently with current computational resources.

In contrast, quantum cryptography relies on the principles of quantum mechanics, such as superposition and the observer effect. Its security is derived from physical laws rather than computational assumptions.

2. Security Basis

In classical systems, security is **computationally secure**, meaning that encryption can theoretically be broken if sufficient computational power is available.

Quantum cryptography provides **information-theoretic security**, where any attempt to intercept the communication can be detected due to the disturbance caused in the quantum system.

3. Key Distribution

Key distribution is a major challenge in classical cryptography. Secure channels or complex protocols are required to exchange encryption keys safely.

Quantum cryptography, particularly through Quantum Key Distribution (QKD), enables secure key exchange over an insecure channel. Any eavesdropping attempt alters the quantum states, alerting the communicating parties.

4. Resistance to Quantum Attacks

Classical cryptographic algorithms, especially widely used public-key systems, are vulnerable to future quantum computing attacks. Quantum algorithms could significantly reduce the time required to break these systems.

Quantum cryptography is inherently resistant to quantum attacks, as its security does not depend on computational difficulty but on quantum physical properties.

5. Implementation Requirements

Classical cryptography is easy to implement in software and requires no specialized hardware, making it widely used in existing systems.

Quantum cryptography requires specialized hardware such as photon sources, detectors, and quantum channels, making it more complex and expensive to deploy.

6. Performance and Scalability

Classical systems are highly scalable and efficient, suitable for large-scale networks and high-speed communication.

Quantum cryptography currently faces limitations in transmission distance, data rate, and scalability, which restrict its widespread adoption.

7. Maturity of Technology

Classical cryptography is well-established, standardized, and widely deployed across various applications.

Quantum cryptography is still an emerging field, with ongoing research focused on improving practicality and scalability.

8. Summary Table

Feature	Classical Cryptography	Quantum Cryptography
Basis	Mathematical algorithms	Quantum mechanics
Security Type	Computational security	Information-theoretic security
Key Distribution	Complex and vulnerable	Secure via QKD
Quantum Resistance	Vulnerable	Naturally resistant
Implementation	Software-based	Requires specialized hardware
Scalability	High	Limited (currently)
Maturity	Mature and standardized	Emerging technology

1.7 Need for Quantum Key Distribution

The comparison between classical and quantum cryptography highlights a fundamental shift in the approach to securing communication systems. While classical cryptographic techniques have been widely adopted due to their efficiency and scalability, their reliance on computational complexity presents a growing concern. With continuous advancements in computing technologies—particularly the emergence of quantum computing—the foundational assumptions underlying classical encryption methods are increasingly at risk.

One of the most critical challenges in classical cryptography is **secure key distribution**. Even when strong encryption algorithms are used, the overall security of the system depends on how securely the encryption keys are generated, shared, and managed. Traditional key exchange mechanisms often require secure channels or depend on complex mathematical problems, both of which may become vulnerable in the future.

In this context, Quantum Cryptography introduces a fundamentally different solution to the problem of key distribution. **Quantum Key Distribution (QKD)** leverages the principles of quantum mechanics to enable two parties to generate and share a secret key in a manner that is inherently secure. Unlike classical methods, QKD does not rely on computational difficulty but instead ensures security through the physical behavior of quantum systems.

A key advantage of QKD is its ability to **detect eavesdropping**. Any attempt by an unauthorized party to intercept or measure the quantum states used in key transmission introduces disturbances that can be observed by the communicating parties. This feature provides a level of security that is not achievable with classical key distribution techniques.

Furthermore, QKD addresses the long-standing issue of **trust in communication channels**. In classical systems, the security of key exchange often depends on trusted intermediaries or secure infrastructure. QKD eliminates this dependency by allowing secure key exchange even over potentially insecure channels.

Despite its current practical limitations, such as the need for specialized hardware and limited transmission range, QKD represents a significant step toward **future-proof security**. As the demand for secure communication continues to grow—especially in areas involving sensitive and large-scale

data exchange—the importance of robust key distribution mechanisms becomes increasingly evident.

1.8 BB84 Protocol: Detailed Explanation

The BB84 protocol, proposed by Charles H. Bennett and Gilles Brassard in 1984, is the first Quantum Key Distribution (QKD) protocol. It allows two parties, Alice and Bob, to securely generate a shared secret key using quantum mechanics.

Step 1: Bit and Basis Selection

Alice generates:

- Random bits: $b \in \{0,1\}$
- Random bases: $\theta \in \{+, \times\}$

Step 2: Quantum Encoding

Rectilinear (+):

$$0 \rightarrow |0\rangle$$
$$1 \rightarrow |1\rangle$$

Diagonal ($\times$):

$$0 \rightarrow |+\rangle = 1/\sqrt{2} (|0\rangle + |1\rangle)$$
$$1 \rightarrow |-\rangle = 1/\sqrt{2} (|0\rangle - |1\rangle)$$

Step 3: Transmission

Alice sends encoded qubits to Bob.

Step 4: Measurement

Bob chooses random bases θ' and measures:

If $\theta = \theta' \rightarrow$ correct

If $\theta \neq \theta' \rightarrow$ random result

Step 5: Basis Reconciliation

Alice and Bob compare bases and keep only matching ones.

Step 6: Sifted Key

K = bits where $\theta = \theta'$

Step 7: Eavesdropping Detection

QBER = errors / total bits

Step 8: Final Key

Remaining bits form the secure key.

Conclusion:

BB84 ensures secure key exchange using quantum principles, making it resistant to eavesdropping.

2 Literature Review

According to Rui Kang et al. (2025), time-series databases play a crucial role in managing and analyzing large volumes of IoT data generated by sensor networks. The study focuses on improving the efficiency of data processing by utilizing advanced encoding techniques, including Delta encoding, Repeat encoding, and Packing methods, which together achieve higher compression ratios and reduce storage overhead and latency.

The authors highlight that while efficient data compression reduces input/output operations, it introduces challenges in query execution, particularly due to the need for decoding before processing. To address this issue, the paper proposes optimized operators that enable direct aggregation on encoded data without full decoding. These operators support parallel processing through thread-level and instruction-level optimizations, including SIMD vectorization. The approach is implemented within the Apache IoTDB framework, where a pipeline-based query engine is developed to accelerate selective aggregation tasks such as down-sampling.

Experimental results demonstrate significant improvements in query performance and computational efficiency compared to existing methods. However, the proposed approach primarily focuses on data processing efficiency and does not address security aspects such as data encryption or protection against unauthorized access, which remain critical challenges in IoT systems.

According to Zakia Zaman et al. (2023), privacy preservation in IoT data analysis is a critical challenge due to the sensitive nature of user data collected by smart systems such as healthcare and smart cities. The study explores probabilistic data encoding techniques, particularly Bloom Filter (BF) encoding combined with Differential Privacy (DP), to secure data before it is processed using machine learning and deep learning algorithms.

The authors propose an enhanced encoding approach by integrating Bloom Filter encoding with Local Differential Privacy (LDP) to improve both privacy and data utility. This method is applied to various datasets, including image data and time-series IoT data, demonstrating its versatility across different domains. The encoded data is then used in deep learning models for tasks such as classification and prediction. The results show a significant improvement of approximately 30% in classification accuracy compared to traditional BF encoding methods. Additionally, the study evaluates the robustness of the approach against Membership Inference Attacks (MIA), where the results indicate that attackers are limited to random guessing with an accuracy of around 50%, thereby enhancing model privacy. However, despite improvements in privacy-preserving data encoding and machine learning performance, the approach does not address real-time secure communication or encryption during data transmission, which remains a crucial requirement in IoT systems.

According to Jakub Hyla et al. (2021), Low-Density Parity-Check (LDPC) codes provide highly efficient error-correction performance and can be adapted for use in resource-constrained IoT systems. The study focuses on implementing an efficient LDPC encoding algorithm on microcontroller-based devices, where computational power and memory are limited. The proposed method optimizes memory usage by efficiently handling the parity-check matrix and auxiliary computations.

The authors demonstrate that the Richardson–Urbanke algorithm can be decomposed into cyclic shifts and GF(2) operations, which align well with basic CPU instructions, making it suitable for low-complexity IoT nodes. Experimental results show significant improvements in memory efficiency and encoding time compared to traditional implementations. Furthermore, when compared with standard block codes such as Reed–Solomon and BCH codes, the proposed LDPC approach achieves better error-correction performance while maintaining comparable memory requirements and reduced energy consumption. However, the work primarily focuses on improving communication reliability through error correction and does not address data security aspects such as encryption or protection against cyber threats, which are essential in IoT environments.

According to B. Gnaneshwari (2025), the Internet of Things (IoT) ecosystem faces significant privacy and security challenges due to device heterogeneity, dynamic behavior, and evolving cyber threats. The study proposes a novel encoding mechanism called Replacement Encoding (RE) to protect sensitive data during AI model training while maintaining data utility and model performance.

The proposed RE technique camouflages data and integrates automated preprocessing to enhance AI-based attack detection. Message packet features extracted from the CICIoT2023 dataset using network analysis tools are processed with the RE method and applied to machine learning models such as Random Forest and Deep Neural Networks. The results demonstrate classification accuracies of 88.94% (RF) and 86.61% (DNN) using 100 features. Additionally, optimized feature selection methods achieve higher accuracies, highlighting the importance of efficient feature engineering. The approach also shows applicability across advanced AI systems, including transformer-based models. However, the proposed method primarily focuses on privacy-preserving data encoding and AI-based threat detection, and does not address secure real-time data transmission or encryption mechanisms, which are critical for end-to-end IoT security.

According to Insik Yoon et al., a hardware-based cryptosystem implementing Homomorphic Encryption was developed to enable secure computation directly on encrypted data for IoT applications. The study presents a 55nm test-chip prototype capable of performing both encryption and decryption operations with high energy efficiency.

The proposed crypto-engine demonstrates low power consumption, requiring approximately 50 nJ for encoding and 13 nJ for decoding, making it suitable for resource-constrained sensor nodes and IoT devices. This approach allows data to remain encrypted even during processing, thereby enhancing data privacy and security in distributed environments. The implementation

highlights the feasibility of integrating advanced cryptographic techniques into hardware for real-time secure computation.

However, the system focuses primarily on computational security and hardware efficiency, and does not address aspects such as key distribution, network-level security, or integration with lightweight IoT communication protocols.

According to Aarthi Gopalakrishnan et al. (2025), the rapid expansion of the Internet of Things (IoT) has increased the need for robust security mechanisms, particularly in intrusion detection systems. The study proposes a novel deep learning-based framework called DEBIT (Deep Learning-Based Intrusion Detection Technique), which efficiently identifies malicious activities in IoT networks.

The proposed method utilizes a gated dilated causal convolution-based encoder-decoder (GDCC-ED) model to preprocess and classify IoT data into attack and non-attack categories. The system is evaluated using a MATLAB simulation environment and assessed based on performance metrics such as accuracy, recall, detection rate, precision, and F1 score. Experimental results demonstrate that the DEBIT framework achieves an accuracy of approximately 99.5%, significantly outperforming existing methods such as pigeon-inspired optimization, Res-TranBiLSTM, and blockchain-based African buffalo algorithms. However, the approach primarily focuses on intrusion detection using deep learning techniques and does not incorporate data encryption or secure communication mechanisms, which are essential for comprehensive IoT security.

According to Narayana Rao Kumana et al. (2024), ensuring data security in IoT systems is a major challenge due to continuous data transmission and increasing cyber threats. The study proposes an integrated approach that combines cryptography and steganography to enhance data protection in IoT environments.

The authors introduce an Elliptic Galois Cryptography protocol to encrypt sensitive data, particularly from medical sources, ensuring confidentiality during transmission. In addition, a Matrix XOR encoding-based steganography technique is employed to embed the encrypted data into low-complexity images, providing an additional layer of security. To optimize the data hiding process, the Adaptive Firefly Algorithm is used for efficient selection of image regions, maintaining visual quality while improving embedding performance. Experimental results demonstrate that the proposed approach effectively enhances data privacy, user authentication, and overall system security. However, the approach involves multiple computational steps, including optimization algorithms, which may increase system complexity and may not be fully suitable for highly resource-constrained IoT devices requiring lightweight implementations.

According to Zhongguo Yang et al. (2021), IoT systems that rely on deep learning models are vulnerable to adversarial attacks, where small perturbations in time-series data can lead to incorrect predictions and unreliable system behavior. The study proposes a robust IoT data

security model using thermometer encoding combined with an encode-decode joint training framework to enhance resistance against such attacks.

The proposed method applies thermometer encoding as a nonlinear transformation to the original data, followed by reconstruction using an encoder-decoder model. A ResNet-based classifier is then trained on the reconstructed data to improve robustness. Experimental results show that the model can effectively resist adversarial attacks, particularly the Fast Gradient Sign Method (FGSM), thereby improving the reliability and security of IoT time-series data classification. However, the approach primarily focuses on enhancing model robustness against adversarial attacks and does not address secure data transmission or encryption mechanisms, which are essential for comprehensive IoT security.

According to Manju Khari et al., ensuring data security in IoT environments is a major challenge due to continuous data transmission and increasing privacy concerns. The study proposes a combined approach using cryptography and steganography techniques to enhance data protection and user authentication.

The authors introduce an elliptic Galois cryptography protocol to encrypt sensitive data obtained from various medical sources. The encrypted data is then embedded into low-complexity images using a Matrix XOR encoding-based steganography technique. To further improve efficiency, an Adaptive Firefly optimization algorithm is used to select optimal cover image regions for embedding, ensuring minimal distortion and effective data hiding. Experimental results demonstrate that the proposed approach improves data security, privacy, and robustness compared to existing methods. However, the system involves multiple computational processes, including encryption, steganography, and optimization, which may increase complexity and limit its applicability in highly resource-constrained IoT devices.

According to Liang Xiao et al., securing IoT systems requires intelligent detection of anomalies due to the dynamic and distributed nature of devices. The study proposes a machine learning-based security framework that analyzes network behavior to detect potential attacks. By using feature extraction and classification techniques, the system improves detection accuracy and reduces false positives. However, the approach mainly focuses on detection and does not provide mechanisms for secure data encryption or transmission.

According to Wei Wang et al., deep learning techniques can significantly enhance IoT security by identifying complex attack patterns in network traffic. The study utilizes convolutional neural networks (CNN) for intrusion detection, achieving high accuracy in identifying malicious activities. Despite its effectiveness, the model requires high computational resources, making it less suitable for deployment on lightweight IoT devices.

According to Al-Sakib Khan Pathan, IoT security faces challenges due to the lack of standardized protocols and weak authentication mechanisms. The research highlights various threats such as denial-of-service attacks, data breaches, and unauthorized access. It emphasizes the need for strong encryption and authentication techniques to ensure secure communication.

However, the study remains theoretical and does not propose a specific implementation framework.

According to Sicong Liu et al., blockchain technology can be integrated with IoT systems to enhance data integrity and security. The decentralized nature of blockchain ensures that data cannot be tampered with easily. The study demonstrates improved trust and transparency in IoT networks. However, blockchain introduces latency and scalability issues, which may affect real-time IoT applications.

According to Xiaofeng Chen et al., cloud-based IoT systems require secure data outsourcing mechanisms to protect sensitive information. The authors propose encryption-based secure storage and access control techniques to prevent unauthorized access. While the approach improves data confidentiality, it increases computational overhead and may not be efficient for real-time systems.

According to Keke Gai et al., hybrid security models combining encryption and access control mechanisms can enhance IoT security. The study introduces a multi-layered security architecture that protects data at different stages, including transmission, storage, and processing. Although effective, the system complexity increases, making implementation challenging in large-scale IoT environments.

3. Methodology

3.1 BB84 Protocol

The BB84 protocol is the first and most fundamental Quantum Key Distribution (QKD) scheme, proposed in 1984 by Charles H. Bennett and Gilles Brassard. It provides a secure method for two parties, traditionally referred to as Alice (sender) and Bob (receiver), to generate a shared secret key using the principles of quantum mechanics. Unlike classical key distribution techniques, the security of BB84 is not based on computational difficulty but on the fundamental laws of physics.

The protocol relies on the behavior of quantum bits, or qubits, which can exist in superposition states. In BB84, information is encoded in the polarization states of photons, using two different sets of bases: the **rectilinear basis** (+) and the **diagonal basis** (×). The rectilinear basis represents binary values using horizontal and vertical polarizations, while the diagonal basis uses 45° and 135° polarizations.

The BB84 protocol begins with Alice generating two random sequences: a sequence of binary bits and a corresponding sequence of bases. Each bit is encoded into a quantum state based on the selected basis. For example, in the rectilinear basis, a bit value of 0 may be represented as a horizontal polarization state, while a bit value of 1 is represented as a vertical polarization state. In the diagonal basis, the same bits are represented using diagonal polarizations. This encoding ensures that the same bit value can have different physical representations depending on the basis used.

Once the qubits are prepared, Alice transmits them to Bob through a quantum communication channel. Upon receiving each qubit, Bob independently and randomly selects a measurement basis (either rectilinear or diagonal) and performs a measurement. Due to the properties of quantum mechanics, if Bob chooses the same basis as Alice, he obtains the correct bit value. However, if he chooses a different basis, the measurement result becomes random, introducing uncertainty.

After the transmission is complete, Alice and Bob communicate over a classical public channel to compare the bases they used for each qubit. Importantly, they do not reveal the actual bit values, only the bases. They then discard all bits where their chosen bases do not match. The remaining bits, where both used the same basis, form what is known as the **sifted key**.

To ensure the security of the communication, Alice and Bob perform an error-checking step. They randomly select a subset of the sifted key and compare their values. If the error rate is within an acceptable threshold, the communication is considered secure. However, if the error rate is significantly high, it indicates the possible presence of an eavesdropper, often referred to as Eve. In such cases, the key is discarded, and the process is repeated.

The ability to detect eavesdropping is a unique and powerful feature of the BB84 protocol. According to quantum mechanics, any attempt by an eavesdropper to measure the transmitted qubits will disturb their states due to the observer effect. Since Eve does not know the correct basis for each qubit, her measurements introduce errors in the system, which can be detected during the error-checking phase.

After confirming the absence of eavesdropping, the remaining bits are further processed using classical techniques such as error correction and privacy amplification to produce the final secret key. This key can then be used for secure communication using symmetric encryption methods.

One of the major advantages of the BB84 protocol is that it provides **information-theoretic security**, meaning that its security is guaranteed by physical laws rather than computational assumptions. This makes it resistant to attacks from both classical and quantum computers. However, practical implementation of BB84 requires specialized hardware such as single-photon sources and detectors, and it may be affected by noise, signal loss, and distance limitations.

3.2 Image-Based Data Extraction Using LSB Steganography

In addition to encrypted data transmission, the system incorporates a secondary mechanism for secure communication using **image-based steganography**. This method enables the embedding and extraction of hidden textual information within digital images, thereby enhancing security by concealing the existence of the data itself. Unlike traditional encryption, which protects the content of a message, steganography focuses on hiding the presence of the message.

The technique employed in this system is based on the **Least Significant Bit (LSB) substitution method**, one of the most widely used approaches in image steganography. In a digital image, each pixel is represented by bytes, and each byte consists of 8 bits. The least significant bit (LSB) contributes minimally to the overall visual representation of the image. Therefore, modifying this bit does not produce noticeable changes in image quality, making it suitable for embedding hidden data.

In the embedding process, the secret message is first converted into its binary representation. Additionally, the length of the message is encoded into the first 16 bits to facilitate proper extraction at the receiver side. Each bit of the message is then embedded into the LSB of consecutive image bytes. Mathematically, this can be represented as modifying each byte such that its least significant bit matches the corresponding message bit. This process results in a modified image that visually appears identical to the original but contains hidden information.

Once the embedding is completed, the image data is converted into a hexadecimal format and transmitted over a network using a socket connection. At the receiver side, the server reconstructs the image by converting the received hexadecimal data back into byte format and storing it as an image file. This ensures that the original structure of the image is preserved during transmission.

The extraction process involves reading the image byte stream and retrieving the least significant bit from each byte. To ensure reliable decoding, an offset is used to skip the initial portion of the image data, which may contain headers or metadata that should not be altered. The extracted bits are then grouped to reconstruct the original message. The first 16 bits are interpreted as the length of the hidden message, and the subsequent bits are grouped into 8-bit segments to form characters, which are finally converted into readable text.

This method provides an additional layer of security by combining data hiding with communication. Even if the transmission is intercepted, the presence of hidden information is not easily detectable.

without knowledge of the embedding technique and extraction process. However, the approach has certain limitations, such as sensitivity to image compression and restricted data capacity.

3.3 Block Diagram Description

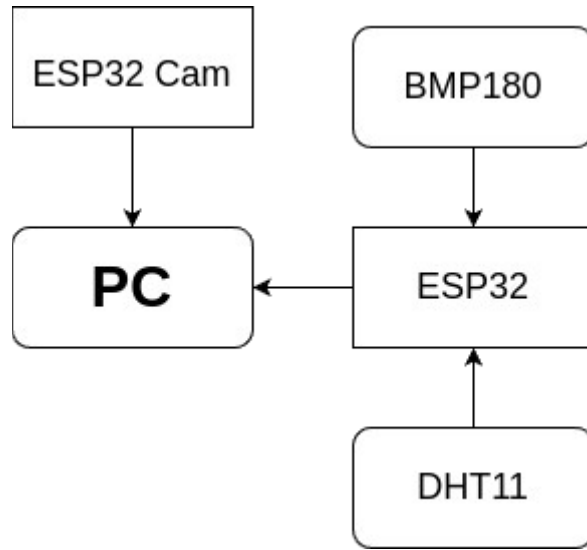


Fig 3.1: Block Diagram

The proposed system consists of multiple hardware components and a central processing unit that work together to perform secure data acquisition, transmission, and processing.

At the sensing level, the system uses environmental sensors such as the **DHT11** and **BMP180**. The DHT11 sensor is responsible for measuring temperature and humidity, while the BMP180 sensor measures atmospheric pressure. These sensors are interfaced with the ESP32 microcontroller, which acts as the primary data acquisition unit.

The ESP32 continuously reads data from the connected sensors and processes it. The collected sensor data is then converted into digital form and prepared for secure transmission. In addition to sensor data handling, the ESP32 also manages communication with the ESP32-CAM module when image-based transmission is required.

The ESP32-CAM module is used for capturing images, which are further utilized for steganography-based data embedding. Hidden messages can be encoded within images and transmitted as an additional layer of secure communication.

All processed data, including encrypted sensor values and encoded images, is transmitted over a wireless network to a central system (PC). The communication is established using Wi-Fi, with the ESP32 acting as the client and the PC acting as the server.

The PC serves as the main processing and monitoring unit. It performs multiple functions, including:

- Receiving encrypted sensor data
- Decrypting the data using the shared key
- Extracting hidden messages from received images

- Displaying real-time data through a web-based dashboard

3.4 System Workflow Description

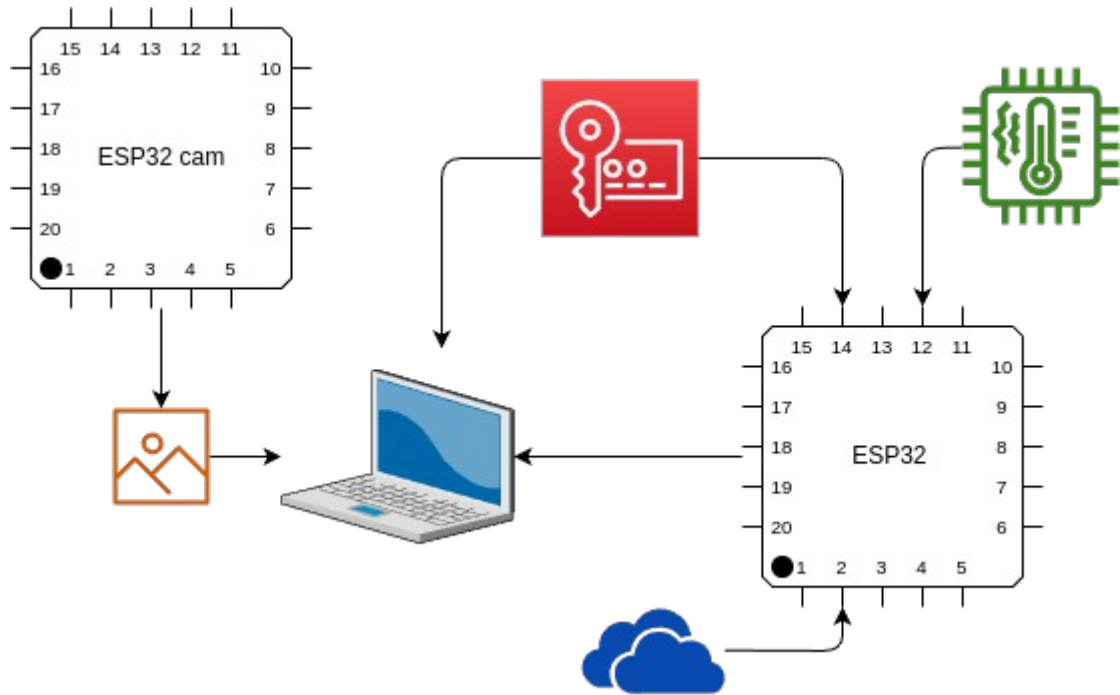


Fig 3.2: Workflow

The proposed system follows a structured workflow that integrates IoT sensing, encryption, and steganography to ensure secure data transmission. The workflow begins at the sensing and data acquisition stage and progresses through multiple security layers before reaching the final processing unit.

Initially, environmental data is collected using sensors connected to the ESP32 microcontroller. The DHT11 sensor measures temperature and humidity, while the BMP180 sensor captures atmospheric pressure. The ESP32 processes these sensor readings and converts them into a digital format suitable for further operations.

Once the data is acquired, it undergoes preprocessing, where values are scaled and converted into binary form. This binary data is then encrypted using a lightweight XOR-based encryption mechanism. The encryption key is generated using a Quantum Key Distribution (QKD)-inspired approach, ensuring secure and dynamic key generation.

Parallel to the sensor data processing, the ESP32-CAM module is used for image handling. In this stage, additional data (such as text messages) is embedded into an image using the Least Significant Bit (LSB) steganography technique. This allows hidden information to be transmitted covertly within image data, adding an extra layer of security.

After encryption and/or steganographic embedding, the data is transmitted over a wireless network using Wi-Fi. The ESP32 acts as the transmitting node, sending either encrypted sensor data or encoded image data to the receiver system.

At the receiver side, which is a PC-based server, the incoming data is processed based on its type. Encrypted sensor data is decrypted using the same XOR operation and the shared key to recover the original values. These values are then converted back into human-readable form and displayed on a web-based dashboard for real-time monitoring.

For image-based transmission, the server reconstructs the image from the received data and applies an extraction algorithm to retrieve the hidden message. The Least Significant Bits of the image bytes are analyzed to recover the embedded information accurately.

3.5 Software Description

The proposed system integrates multiple software tools and programming environments to implement data acquisition, encryption, quantum key generation, steganography, and visualization. Each software component plays a specific role in ensuring the functionality and security of the system.

3.5.1 MicroPython (ESP32 Programming)

MicroPython is a lightweight implementation of the Python programming language designed for microcontrollers. In this system, MicroPython is used to program the ESP32 module.

The ESP32 uses MicroPython to:

- Interface with sensors such as DHT11 and BMP180
- Collect real-time environmental data
- Perform data preprocessing and scaling
- Convert sensor values into binary format
- Apply XOR-based encryption using the shared key
- Establish Wi-Fi communication and send data to the server

MicroPython is chosen due to its simplicity, low memory usage, and suitability for resource-constrained IoT devices.

3.5.2 Python (Server-Side Processing)

Python is used on the server (PC) side for handling communication, data processing, and visualization.

Its functionalities include:

- Creating a TCP socket server for receiving data
- Decrypting incoming encrypted data
- Reconstructing sensor values from binary format
- Handling image data transmission and decoding
- Running a web server for real-time monitoring

Python provides flexibility and ease of integration with various libraries, making it ideal for rapid development.

3.5.3 Quantum Simulation (QKD Implementation)

A quantum simulation environment is used to implement a BB84-inspired Quantum Key Distribution mechanism.

Key operations include:

- Random bit generation
- Random basis selection
- Quantum circuit creation
- Measurement simulation
- Shared key generation

This simulation enables secure key generation without requiring physical quantum hardware, making it practical for experimental implementation.

3.5.4 Socket Programming

Socket programming is used to establish communication between the ESP32 and the server.

Features:

- TCP-based reliable communication
- Client-server architecture
- Data transmission in binary/string format

The ESP32 acts as a client, while the server continuously listens for incoming data.

3.5.5 Web Server and Dashboard

A lightweight HTTP server is implemented on the PC to visualize data.

Functions:

- Displays temperature, humidity, and pressure
- Provides real-time updates using auto-refresh
- Uses HTML and CSS for user interface design

This enables users to monitor sensor data through a web browser.

3.5.6 Image Processing and Steganography

Python is used for handling image-based communication and steganography.

Operations include:

- Receiving image data in hexadecimal format

- Converting hex data into image bytes
- Saving reconstructed images
- Extracting hidden messages using LSB technique

The Least Significant Bit method is used to retrieve embedded data from the image without affecting its visual quality.

3.5.7 Data Encoding and Decoding

Custom algorithms are implemented for:

- Converting integers to binary bit streams
- Concatenating multiple sensor values
- Encrypting and decrypting data using XOR
- Reconstructing original sensor values

These operations ensure efficient and structured data handling throughout the system.

3.5.8 Development Environment

The system is developed using standard tools such as:

- Code editors for writing MicroPython and Python scripts
- Serial monitor for debugging ESP32
- Local server environment for testing

These tools support efficient coding, debugging, and testing of the system.

3.6 Hardware Description

The proposed system integrates multiple hardware components to perform environmental data acquisition, image processing, and wireless communication. The hardware setup is designed to be cost-effective, efficient, and suitable for real-time IoT applications.

3.6.1 ESP32 Microcontroller

The ESP32 is the core processing unit of the system. It is a low-cost, low-power microcontroller with built-in Wi-Fi and Bluetooth capabilities, making it ideal for IoT applications.

In this system, the ESP32 performs the following functions:

- Interfacing with sensors (DHT11 and BMP180)
- Collecting real-time environmental data
- Processing and scaling sensor values
- Converting data into binary format

- Performing XOR-based encryption
- Establishing Wi-Fi communication with the server

The ESP32 is selected due to its high performance, integrated networking features, and compatibility with MicroPython.

3.6.2 DHT11 Sensor

The DHT11 is a digital sensor used to measure temperature and humidity.

Features:

- Temperature range: 0°C to 50°C
- Humidity range: 20% to 90%
- Digital output signal
- Low power consumption

In this system, the DHT11 provides environmental data that is used for monitoring and secure transmission.

3.6.3 BMP180 Pressure Sensor

The BMP180 is a barometric pressure sensor used to measure atmospheric pressure.

Features:

- High accuracy pressure measurement
- I2C communication interface
- Low power consumption

The sensor communicates with the ESP32 via the I2C protocol and provides pressure data for analysis and transmission.

3.6.4 ESP32-CAM Module

The ESP32-CAM is a compact module that combines an ESP32 microcontroller with a camera.

Functions in the system:

- Captures images
- Supports image-based data transmission
- Enables steganography by embedding hidden messages

This module plays a key role in implementing the image-based communication subsystem.

3.6.5 Power Supply

The system requires a stable power source for proper operation.

- ESP32 operates at 3.3V

- Sensors are powered through the ESP32
- USB or regulated power supply is used

Proper voltage regulation is essential to ensure reliable performance and prevent hardware damage.

3.6.6 Communication Interface

The system uses built-in Wi-Fi capabilities of the ESP32 for communication.

- Protocol: TCP/IP
- Mode: Client (ESP32) → Server (PC)
- Wireless transmission of encrypted data and images

This eliminates the need for additional communication modules.

3.6.7 Interfacing and Connections

The hardware components are connected as follows:

- DHT11 → Digital GPIO pin of ESP32
- BMP180 → I2C interface (SDA, SCL)
- ESP32-CAM → Operates as separate module
- Power supply → Common ground shared across components

Proper wiring ensures accurate data acquisition and stable operation.

3.6.8 System Integration

All hardware components work together as a unified system:

1. Sensors collect environmental data
2. ESP32 processes and encrypts data
3. ESP32-CAM handles image capture and steganography
4. Data is transmitted wirelessly to the PC
5. PC processes and displays results

This integration enables secure and efficient IoT communication.

3.6.9 Advantages of Hardware Design

- Low-cost implementation
- Wireless communication capability
- Modular and scalable design
- Suitable for real-time monitoring
- Supports both data encryption and steganography

3.7 Code Explanation

This section explains the implementation of the proposed system, including sensor data acquisition, encryption, communication, quantum key generation, and steganographic decoding. The code is divided into transmitter-side (ESP32) and receiver-side (server) operations.

3.7.1 Sensor Initialization (ESP32)

The ESP32 initializes sensors using GPIO and I2C interfaces.

```
sensor = dht.DHT11(Pin(4))
```

```
i2c = I2C(scl=Pin(22), sda=Pin(21), freq=100000)
```

```
bmp180 = BMP180(i2c)
```

- DHT11 is connected to a digital pin
- BMP180 uses I2C communication
- These sensors provide temperature, humidity, and pressure

3.7.2 Reading Sensor Data

```
sensor.measure()
```

```
temp_scaled = int(sensor.temperature() * 10 + 0.5)
```

```
hum_scaled = int(sensor.humidity() * 10 + 0.5)
```

```
press_scaled = int((bmp180.pressure / 100) * 10)
```

- Sensor values are read periodically
- Values are scaled ($\times 10$) to preserve decimal precision
- Converted into integers for binary processing

3.7.3 Integer to Binary Conversion

```
def int_to_bits(value, bits=16):
```

```
    out = []
```

```
    for i in range(bits):
```

```
        out.append((value >> (bits - 1 - i)) & 1)
```

```
    return out
```

- Converts integer values into fixed-length (16-bit) binary
- Ensures uniform data size for encryption

3.7.4 Data Formation

```
data_bits = temp_bits + hum_bits + press_bits
```


- Combines all sensor data into a single bitstream
- Total length = 48 bits

3.7.5 XOR Encryption

```
def xor_encrypt(data_bits, key, pos):
```

```
    out = []
```

```
    for b in data_bits:
```

```
        out.append(b ^ key[pos % len(key)])
```

```
        pos += 1
```

```
    return out, pos
```

- Each bit is XORed with key bits
- Key is reused cyclically
- Produces encrypted bitstream

3.7.6 Wi-Fi Connection

```
wifi = network.WLAN(network.STA_IF)
```

```
wifi.active(True)
```

```
wifi.connect(SSID, PASSWORD)
```

- ESP32 connects to wireless network
- Enables communication with server

3.7.7 Data Transmission

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s.connect((SERVER_IP, PORT))
```

```
s.send(send_data.encode())
```

```
s.close()
```

- TCP socket is used
- Encrypted data is sent as string
- Connection is closed after transmission

3.7.8 QKD Key Generation (Server)

```
key_gen = qkd_generation(16)
```

```
key = key_gen.circuit_create()
```

- Generates shared key using BB84 simulation

- Key used for both encryption and decryption

3.7.9 XOR Decryption

```
def xor_decrypt(data_bits, key, pos):
    out = []
    for b in data_bits:
        out.append(b ^ key[pos % len(key)])
        pos += 1
    return out, pos
```

- Same as encryption (XOR is reversible)
- Recovers original data

3.7.10 Data Reconstruction

```
temp_val = int(temp_bits, 2)
temp = temp_val / 10
```

- Binary converted back to integer
- Scaled back to real values

3.7.11 Web Server Dashboard

```
httpd = HTTPServer(("0.0.0.0", 8080), WebHandler)
httpd.serve_forever()
```

- Displays real-time data
- Auto-refresh web interface
- Shows temperature, humidity, pressure

3.7.12 Steganography – Data Extraction

LSB Extraction

```
bits.append(b & 1)
```

- Extracts least significant bit from each byte

Message Length Extraction

```
msg_len = (msg_len << 1) | bits[i]
```

- First 16 bits represent message length

Message Reconstruction

```
byte = (byte << 1) | bits[index]
```

- Groups bits into bytes
- Converts to characters

Final Decoding

`bytes(message_bytes).decode()`

- Converts byte array into readable text

3.7.13 Image Reconstruction

`img_bytes = bytes.fromhex(hex_string)`

with `open(filename, "wb")` as `f`:

`f.write(img_bytes)`

- Converts received hex into image
- Saves file for processing

3.7.14 Overall Flow in Code

Transmitter

1. Read sensor data
2. Convert to binary
3. Encrypt using XOR
4. Send via Wi-Fi

Receiver

1. Generate key
2. Receive data
3. Decrypt
4. Display data
5. Extract hidden message (if image)

4. RESULTS

The proposed system was successfully implemented and tested to evaluate its performance in secure data transmission using IoT, encryption, and steganography techniques. The system integrates environmental sensing, XOR-based encryption with quantum-inspired key generation, and image-based data hiding, demonstrating a multi-layered security approach.

During experimentation, the ESP32 module continuously collected environmental data from the DHT11 and BMP180 sensors. The measured parameters included temperature, humidity, and atmospheric pressure. The collected values were scaled and converted into binary format before being encrypted using a key generated through a simulated Quantum Key Distribution (QKD) mechanism. The encrypted data was then transmitted over a Wi-Fi network to the server system.

At the receiver side, the server successfully received the encrypted data through a TCP socket connection. The decryption process, based on the XOR operation with the shared key, accurately reconstructed the original sensor values. The recovered values matched the transmitted data with minimal error, confirming the correctness and reliability of the encryption and decryption process.

The real-time web dashboard displayed the decrypted sensor values effectively. The interface updated periodically, showing temperature, humidity, and pressure readings in a clear and user-friendly format. This demonstrates the system's capability for real-time monitoring and visualization.

In addition to sensor data transmission, the image-based steganography subsystem was also tested. Images transmitted from the ESP32 were successfully reconstructed at the server. The Least Significant Bit (LSB) extraction algorithm accurately retrieved the hidden message embedded within the image. The decoded messages were consistent with the original input, validating the effectiveness of the steganographic method.

The system showed stable performance under normal network conditions. Data transmission was reliable, with no significant packet loss observed during testing. The encryption process introduced negligible computational overhead, making it suitable for resource-constrained IoT devices like the ESP32.

However, certain limitations were observed. The use of a simulated QKD system means that true quantum-level security is not physically implemented. Additionally, the XOR-based encryption, while lightweight, depends on the quality and reuse of the key. The steganography method is sensitive to image modifications such as compression, which may affect message recovery.

Overall, the results demonstrate that the proposed system effectively achieves secure data transmission by combining encryption and steganography. The integration of IoT devices with quantum-inspired security mechanisms provides a promising approach for enhancing data protection in modern communication systems.

5. CONCLUSION

This project presented the design and implementation of a secure IoT-based data transmission system that integrates environmental sensing, encryption, quantum-inspired key generation, and steganography techniques. The primary objective was to enhance the security of IoT communication by combining lightweight encryption methods with advanced key distribution concepts and covert data transmission.

The system successfully demonstrated real-time acquisition of environmental data using sensors such as DHT11 and BMP180, interfaced with the ESP32 microcontroller. The collected data was processed, converted into binary format, and encrypted using an XOR-based method. A key feature of the system is the use of a Quantum Key Distribution (QKD)-inspired mechanism for generating encryption keys, which adds an additional layer of security compared to conventional static key methods.

The encrypted data was transmitted over a wireless network and successfully received by a server system. The decryption process accurately reconstructed the original sensor values, confirming the reliability of the encryption and communication modules. Furthermore, the implementation of a web-based dashboard enabled real-time visualization of the sensor data, demonstrating the system's effectiveness in monitoring applications.

In addition to encryption, the project incorporated an image-based steganography subsystem using the Least Significant Bit (LSB) technique. This allowed hidden messages to be embedded within images and transmitted covertly. The receiver system successfully extracted the hidden data, validating the effectiveness of the steganographic approach. The combination of encryption and steganography provided a dual-layer security mechanism, significantly improving the confidentiality of transmitted data.